

CVD Platform Overview

A clear, reliable platform with standardized cards for cardiovascular diseases and help clinicians, educators, and public-health analysts tell primary symptoms from secondary factors — and the platform can grow beyond the diploma if needed.

Presented by Darya Korbut

Group: JS/TS

Supervisor: Alevtina Gourinovich

Date: 08.01.2026



Objective and Uniqueness

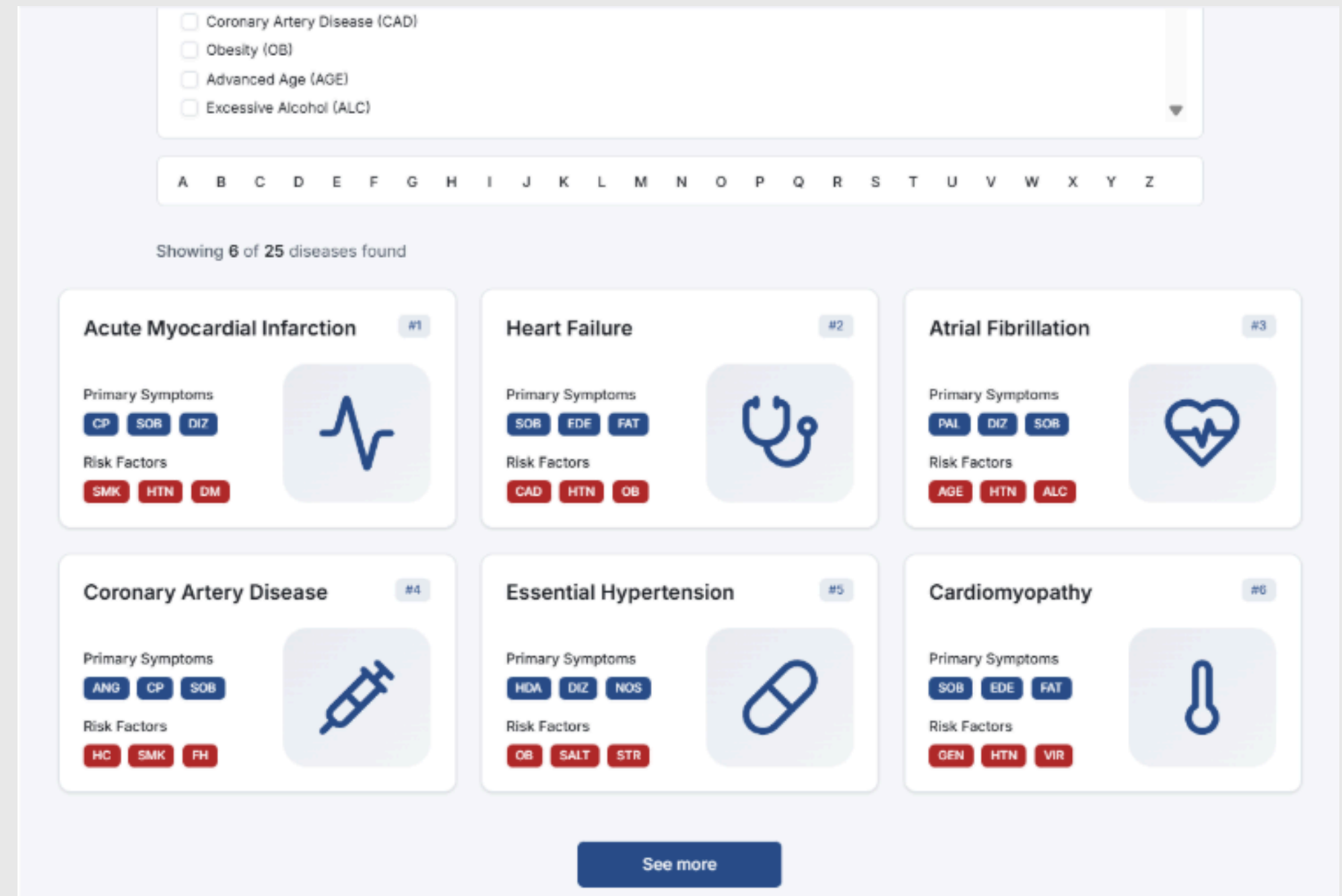
- **Project Objective:** Address the fragmented and unstructured nature of cardiovascular disease (CVD) information by developing a web-based knowledge system that consolidates disease data into standardized, referenced disease cards with consistent descriptions, risk factors, symptoms, and prevention guidance.
- **Uniqueness:** Unlike existing CVD information platforms where databases are not systematically built upon analysis of real clinical datasets, this project establishes its knowledge base (database) through comprehensive analysis of actual patient data from multiple clinical sources.

Problem Statement (BA)

- Today, information on cardiovascular diseases is fragmented, inconsistently structured, and often lacks analytical context.
- Clinicians and educators spend time reconciling disparate sources; public-health analysts lack a unified view of prevalence and trend indicators; and end users struggle to distinguish primary symptoms from secondary factors. This leads to slower decision-making, unclear prevention guidance, and missed insights.
- The platform addresses these issues by unifying disease data in a standardized format, computing prevalence and key indicators from curated datasets, and separating primary vs. secondary factors to improve clarity, comparability, and actionable prevention.

Solution

- **CVD Platform – fast, trustworthy disease knowledge in one place:** A bilingual, searchable CVD knowledge platform that lets users quickly find diseases by symptoms/risk factor, alphabet and explore evidence sources and researches about CVDs.
- The platform combines a user-friendly interface making disease information accessible and engaging for everyone.

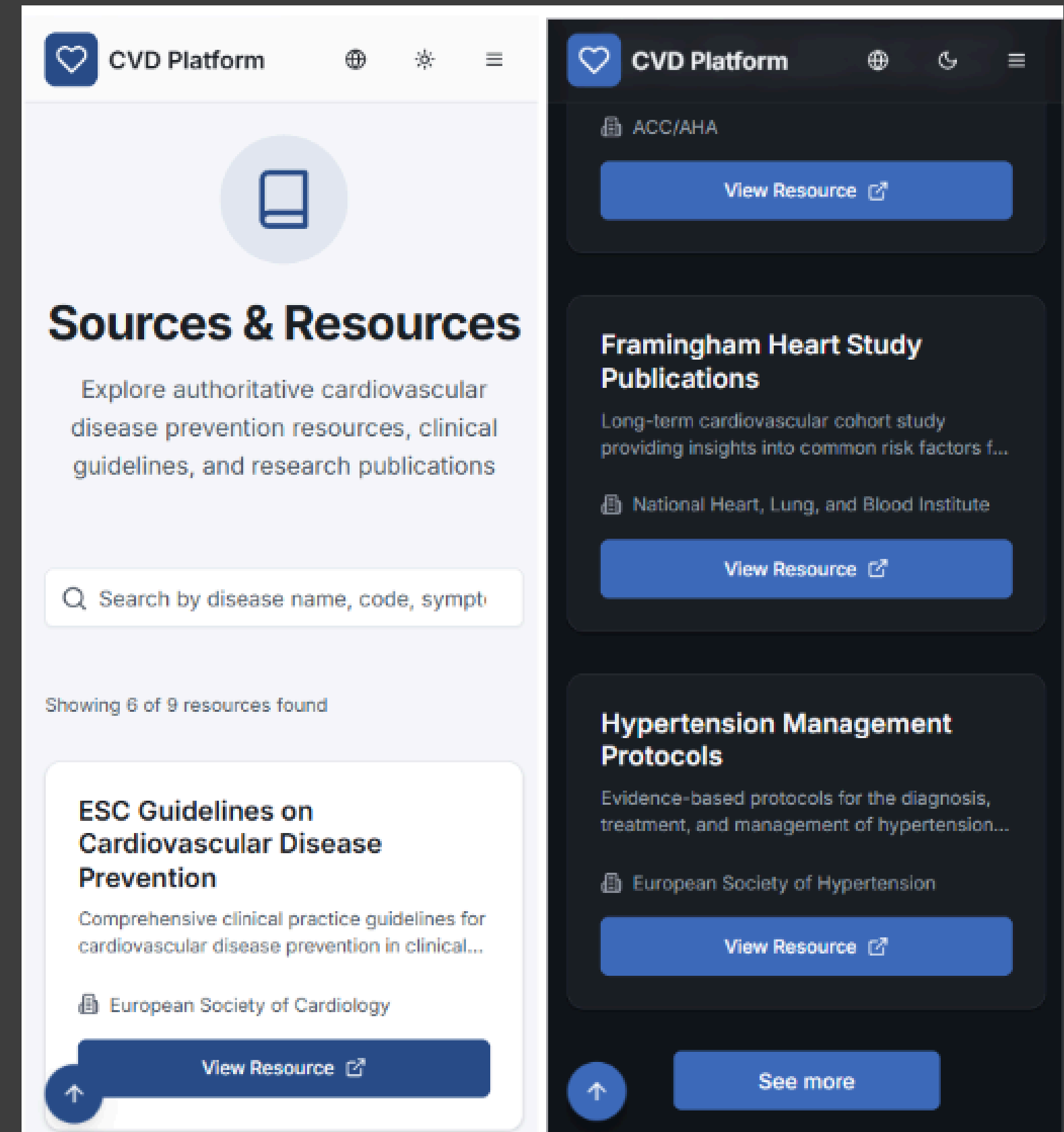


Project Description

- This project focuses on the development of a **web-based knowledge system** for cardiovascular disease prevention. The system combines the following criteria and approaches:
 - A **React frontend** to implement a code solution according to the design
 - An **Express/Prisma backend** to work with the database and create business logic for the platform
 - **PostgreSQL database** to provide standardized, **bilingual** (EN/RU) disease **cards**, inspired by the analyzed raw and real datasets, with consistent descriptions, risk factors, symptoms, prevention guidance and so on.
- The project emphasizes production-ready architecture with **containerization**, comprehensive **testing**, **observability**, and **API-first design** suitable for future expansion.
- The **research component** includes **analysis of real-world** and **raw cardiovascular datasets** (UCI Heart Disease, Statlog, Cleveland, Hungary) containing clinical data from actual patients. This data-driven approach ensures that the prevention recommendations and risk factor information are grounded in validated medical research rather than theoretical assumptions.

Value

- The project carries significant social value by making evidence-based cardiovascular health information accessible to the general public, potentially contributing to early disease awareness and prevention in communities where such knowledge may be limited.



Key Features

CVD Cards

- Quick card view of each cardiovascular disease
- Key info at a glance: main symptoms and risk factors
- Click a card to open the full disease page

Smart Search & Filters

- Search by disease name, code or keywords
- Filter results by primary and secondary symptoms and risk factors
- "Reset" button in filters to reset and explore again
- Filter the cards by alphabet

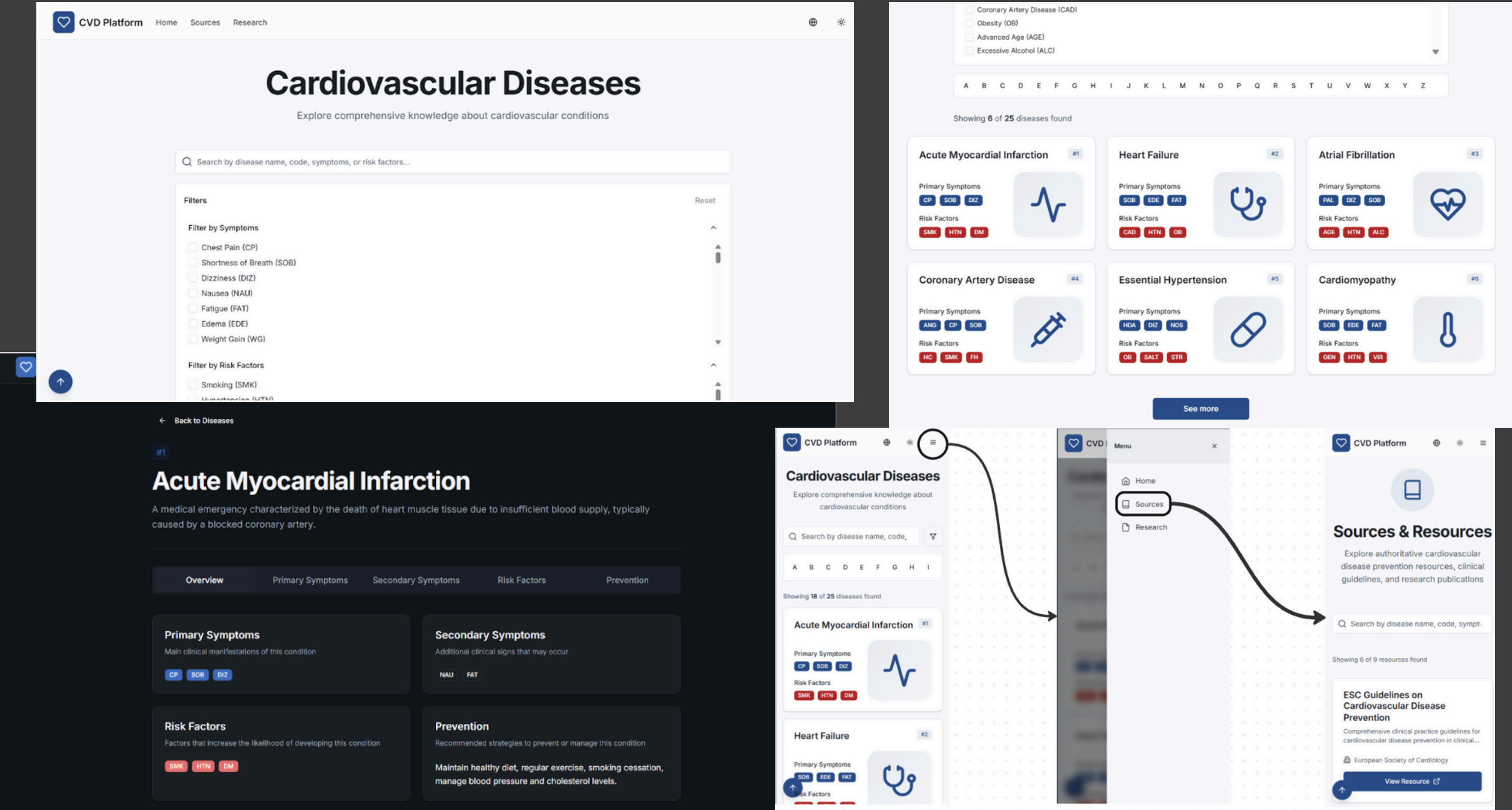
Sources & Researches

- Verified references linked to each topic
- Research library for deeper reading and evidence

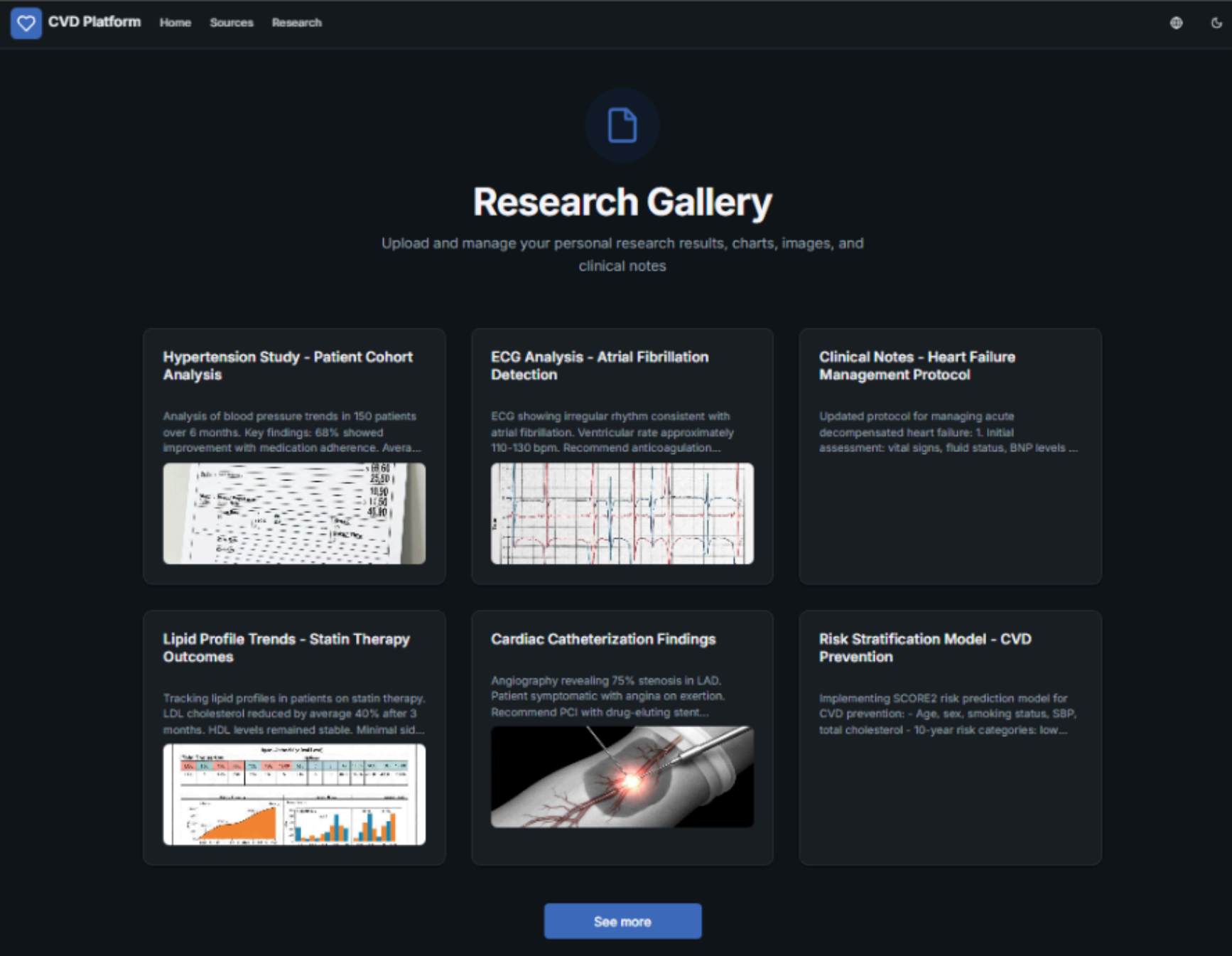
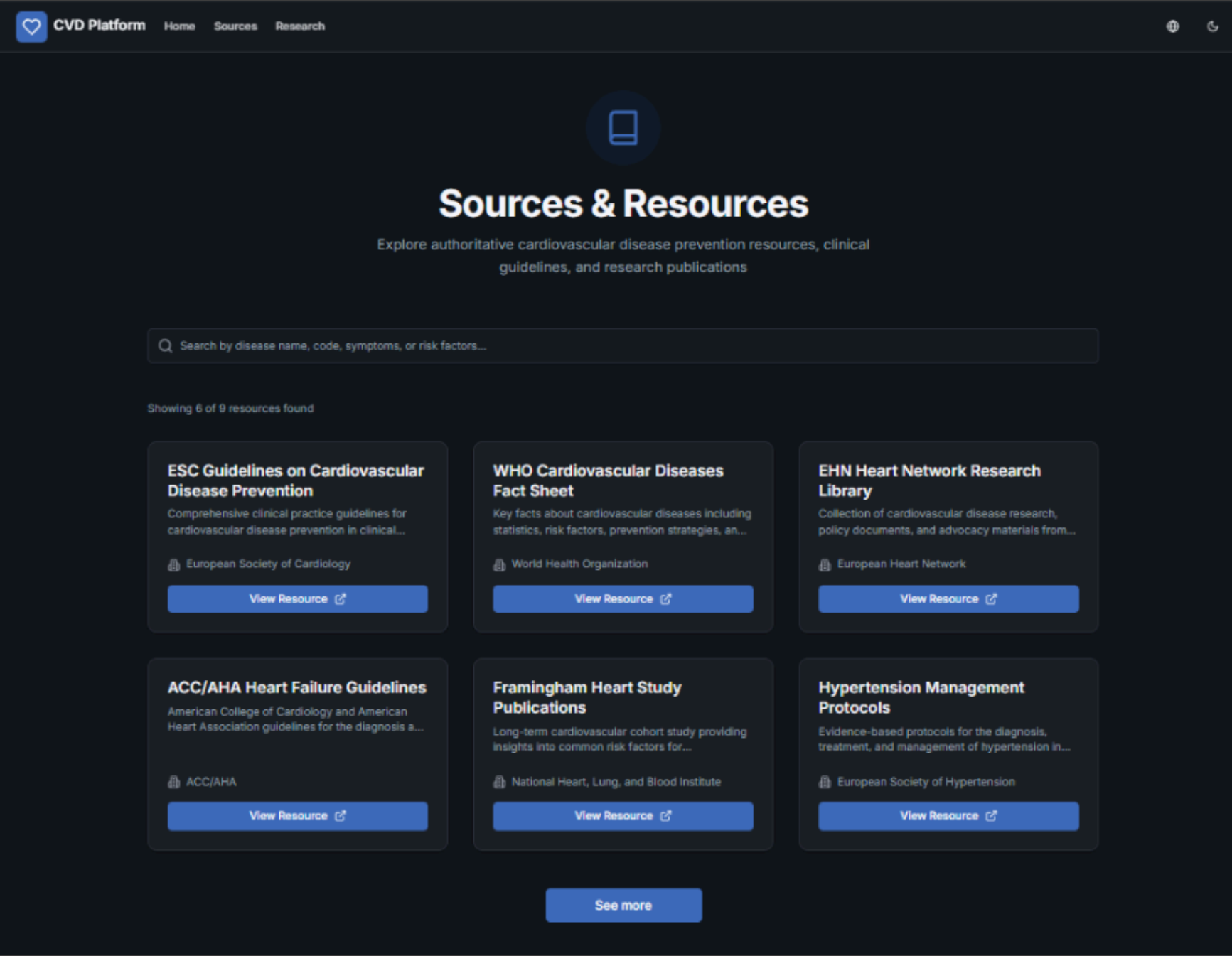
Usability & Accessibility

- Switch between English and Russian anytime
- Toggle Light / Dark theme
- Scroll-to-top button for long pages
- Full keyboard navigation (no mouse needed)

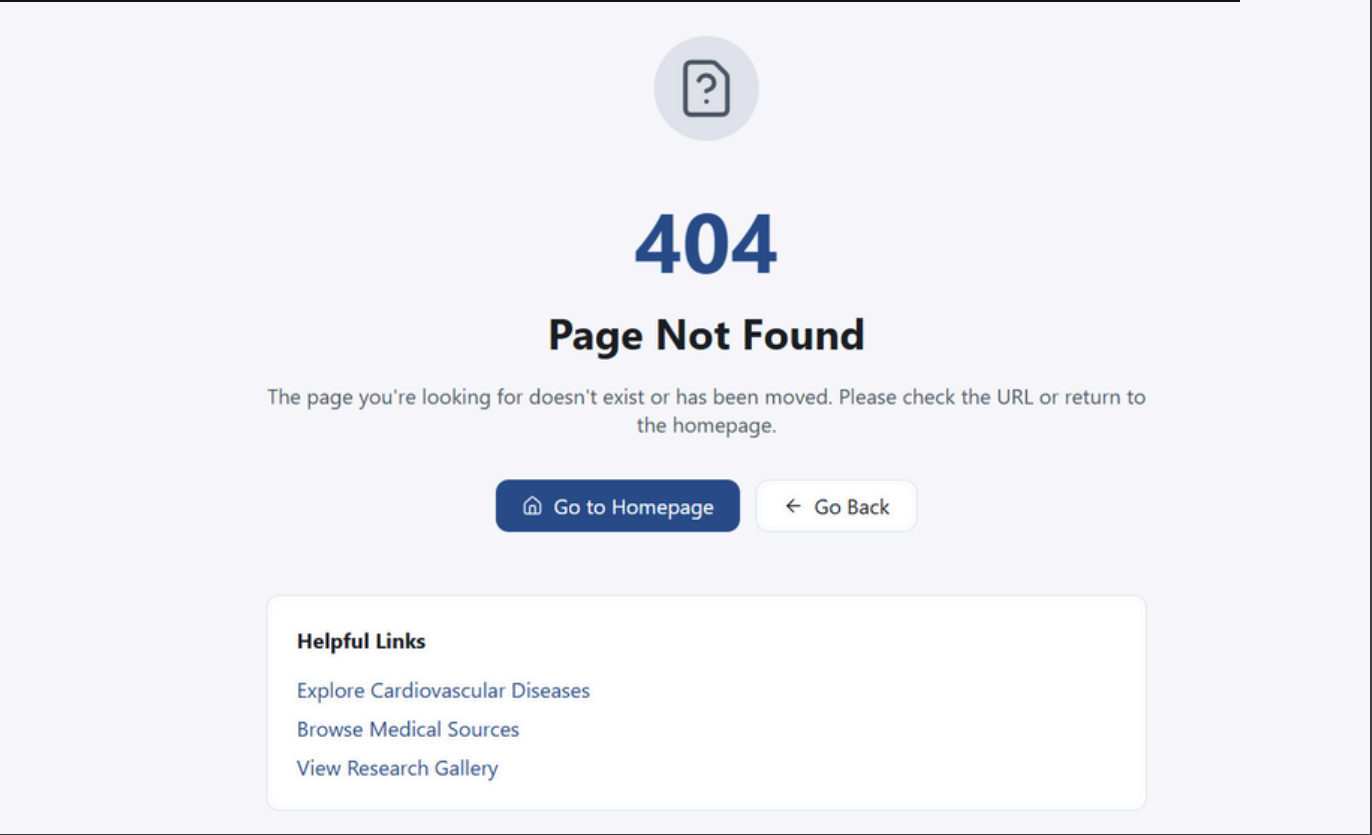
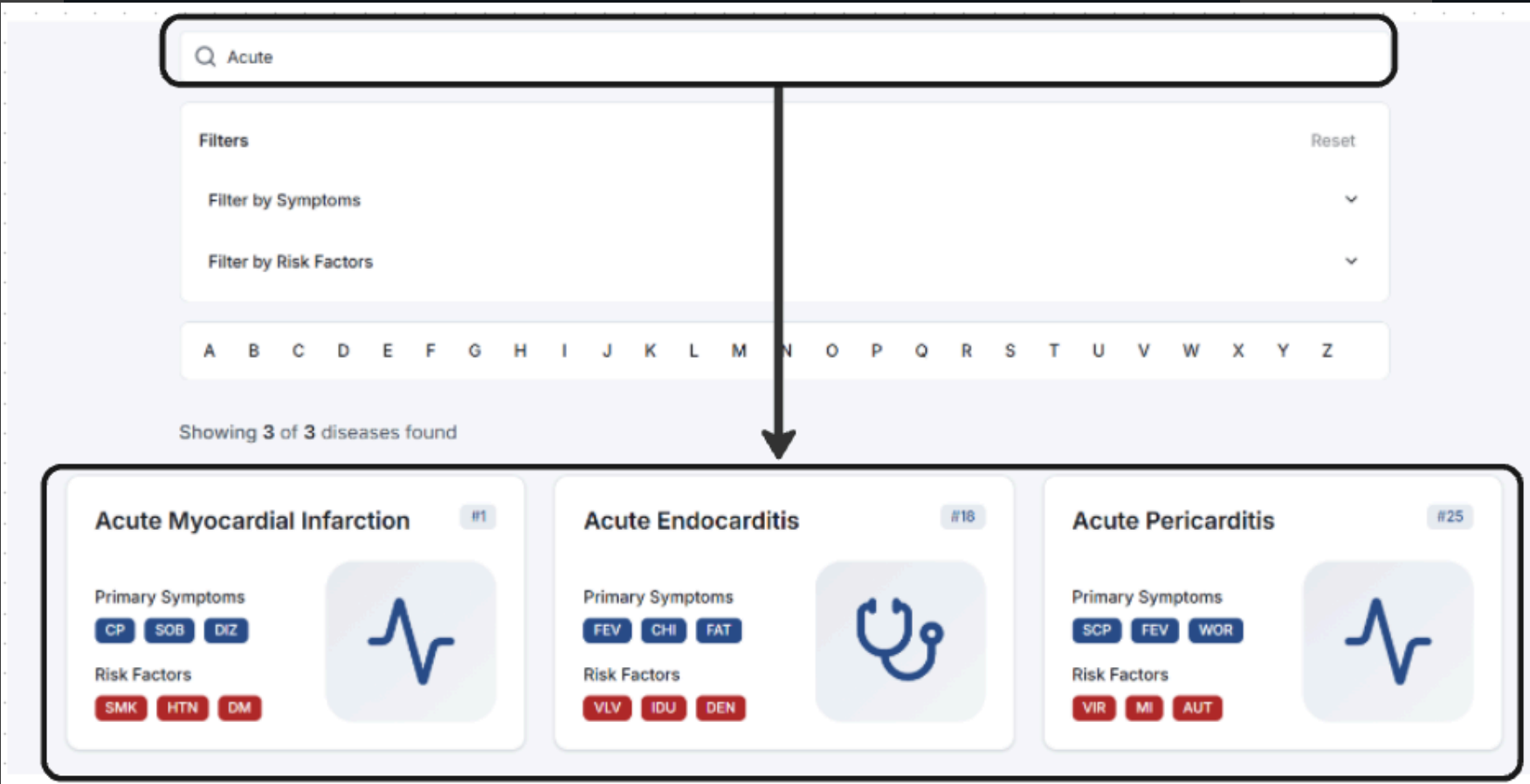
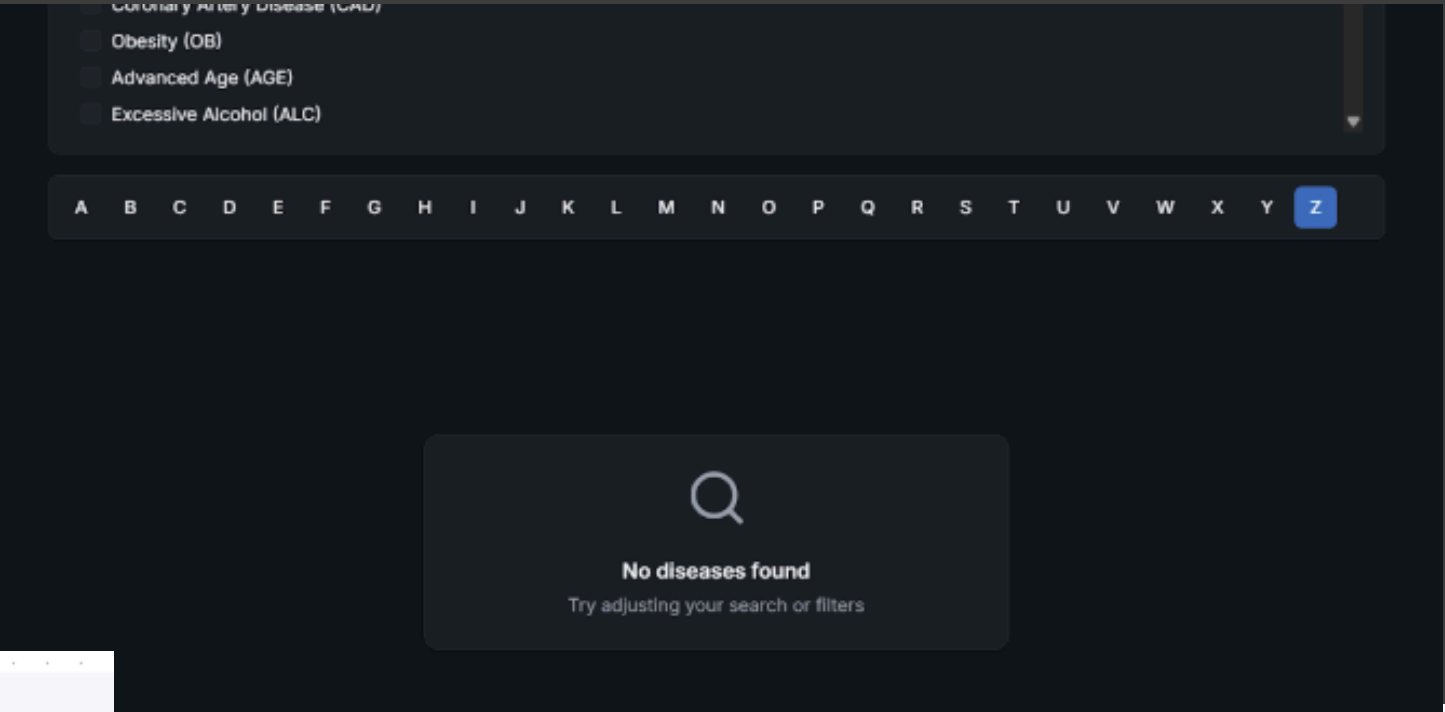
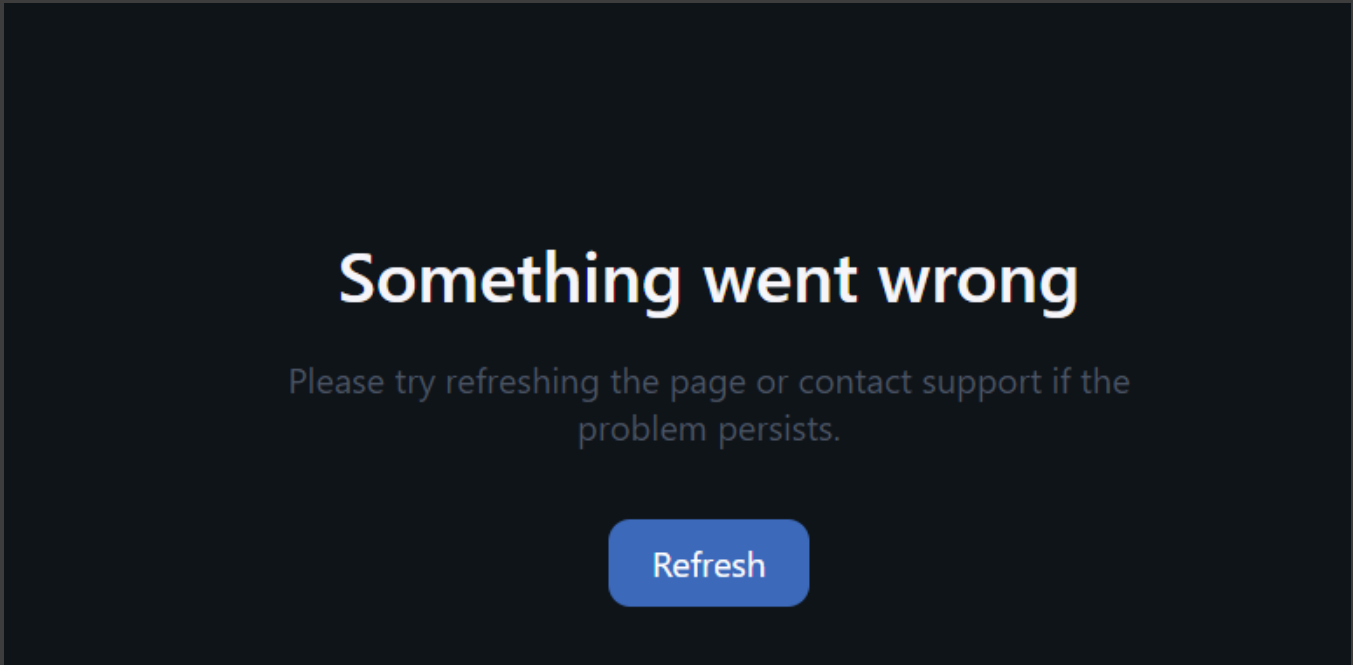
Main and Disease Pages



Source and Research Pages

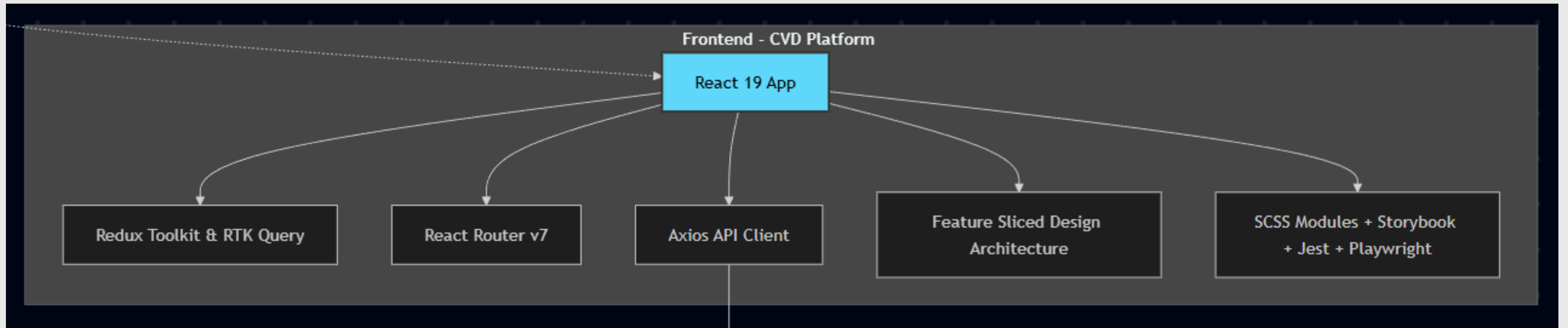


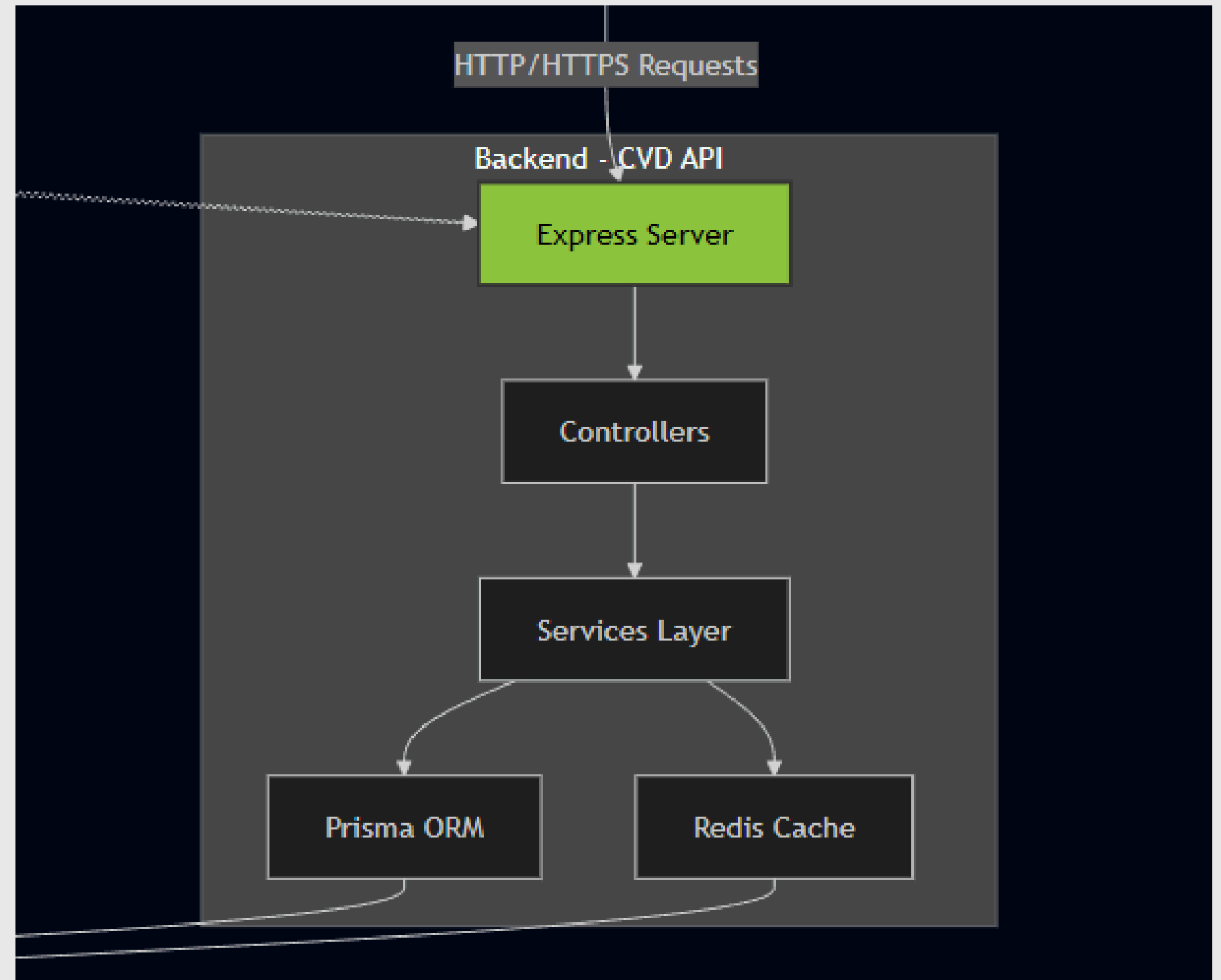
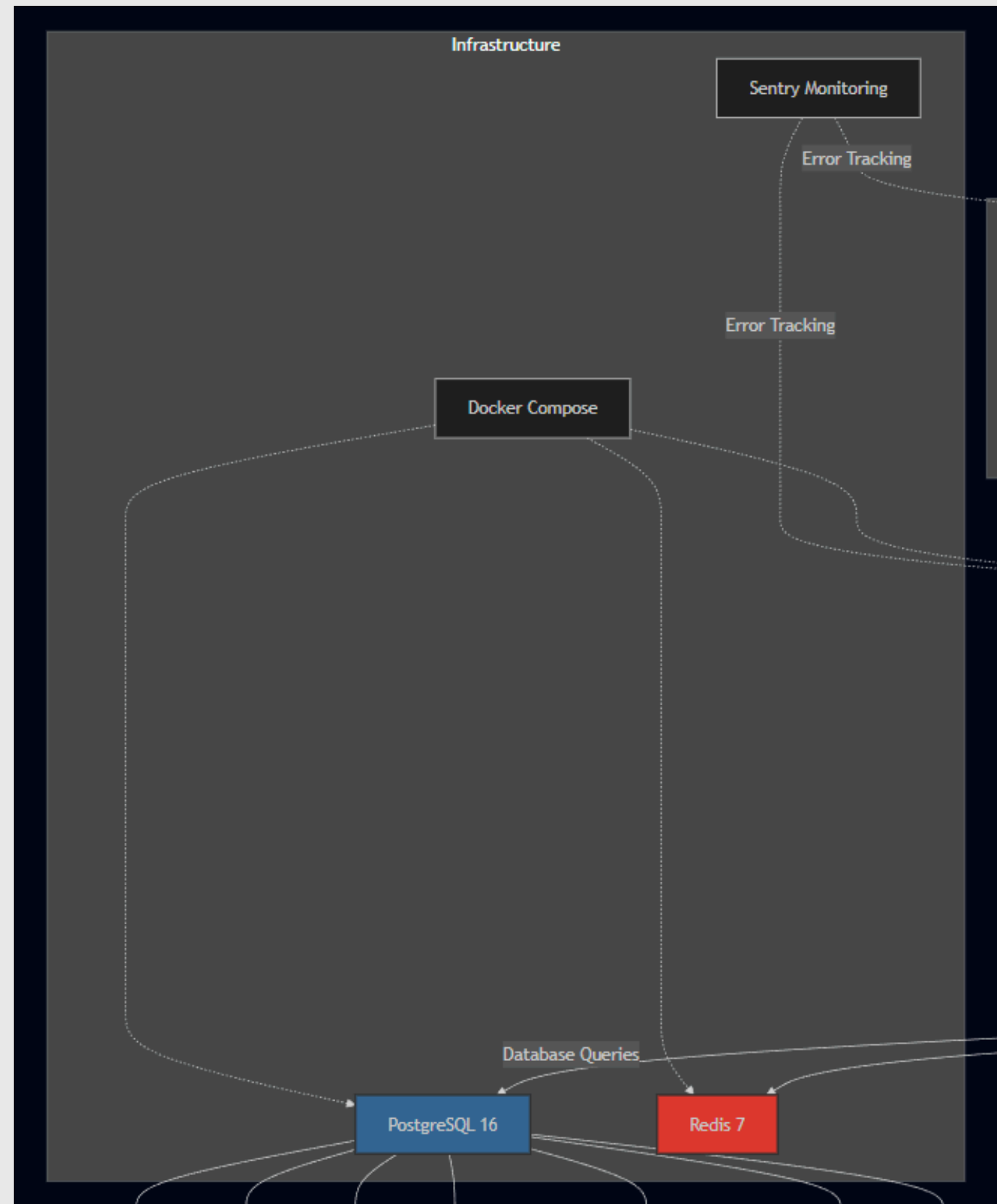
UI States

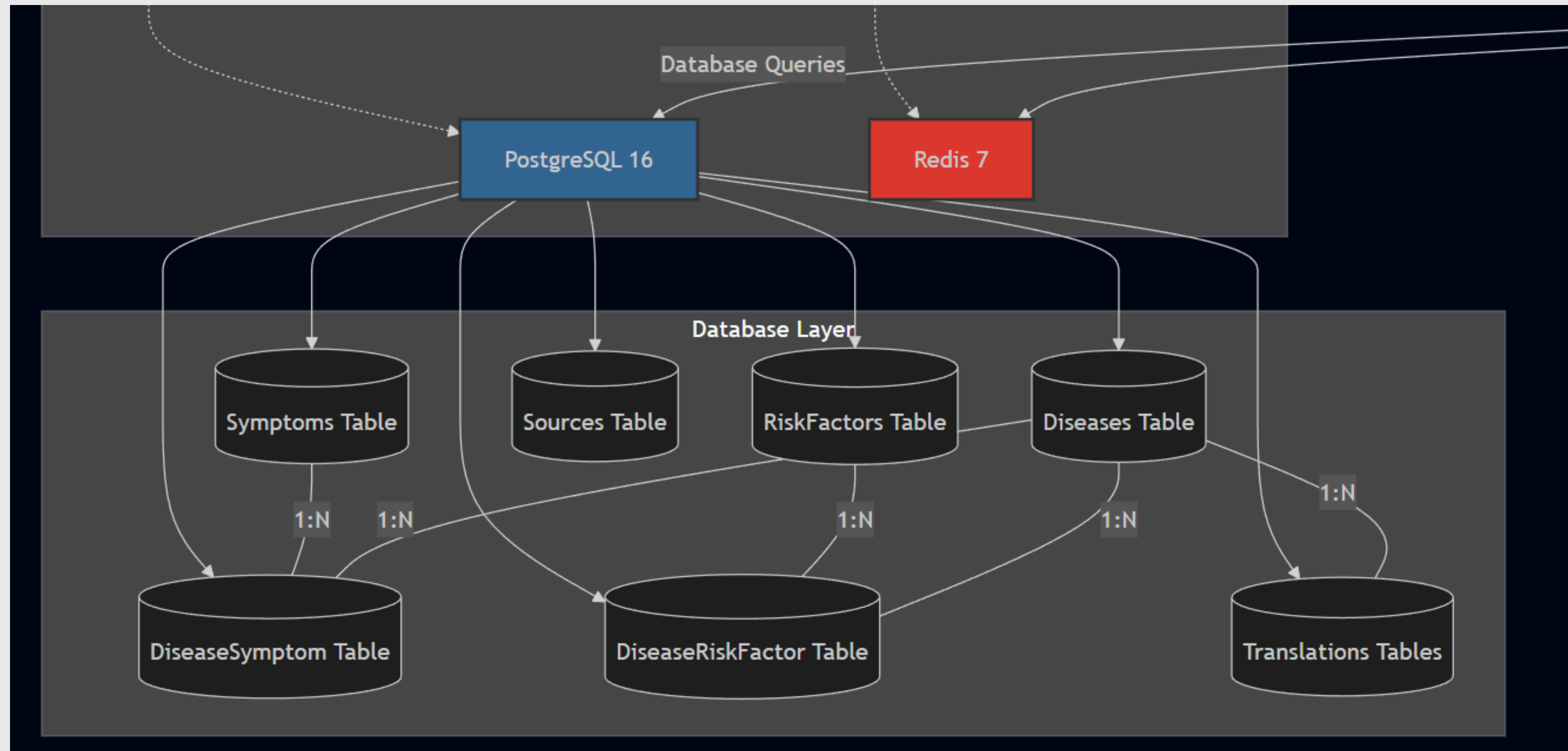




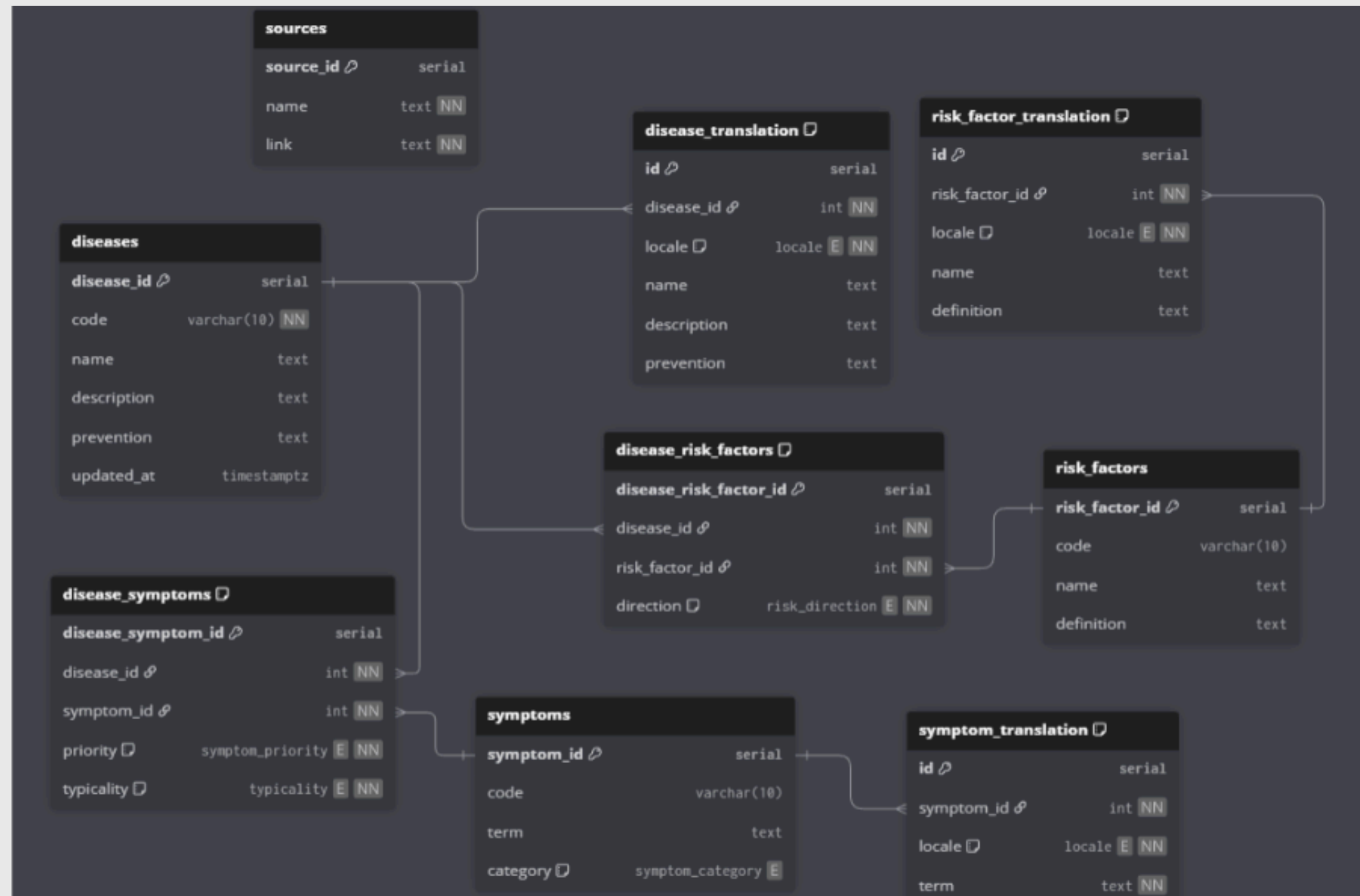
High Level Architecture







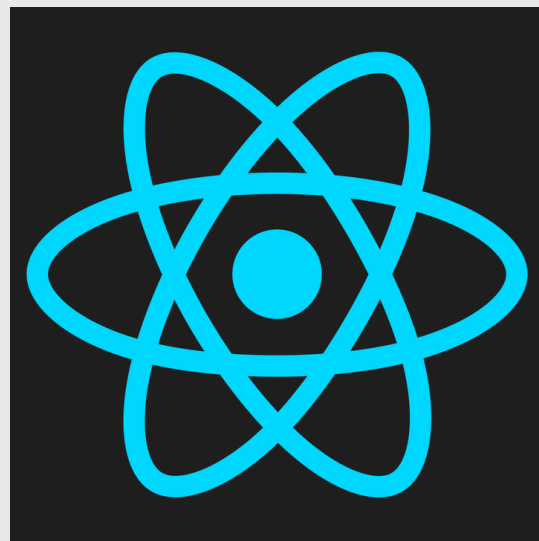
Database Relationship Diagram



Backend Technology Stack



Frontend Technology Stack



Frontend Architecture & UX Delivery

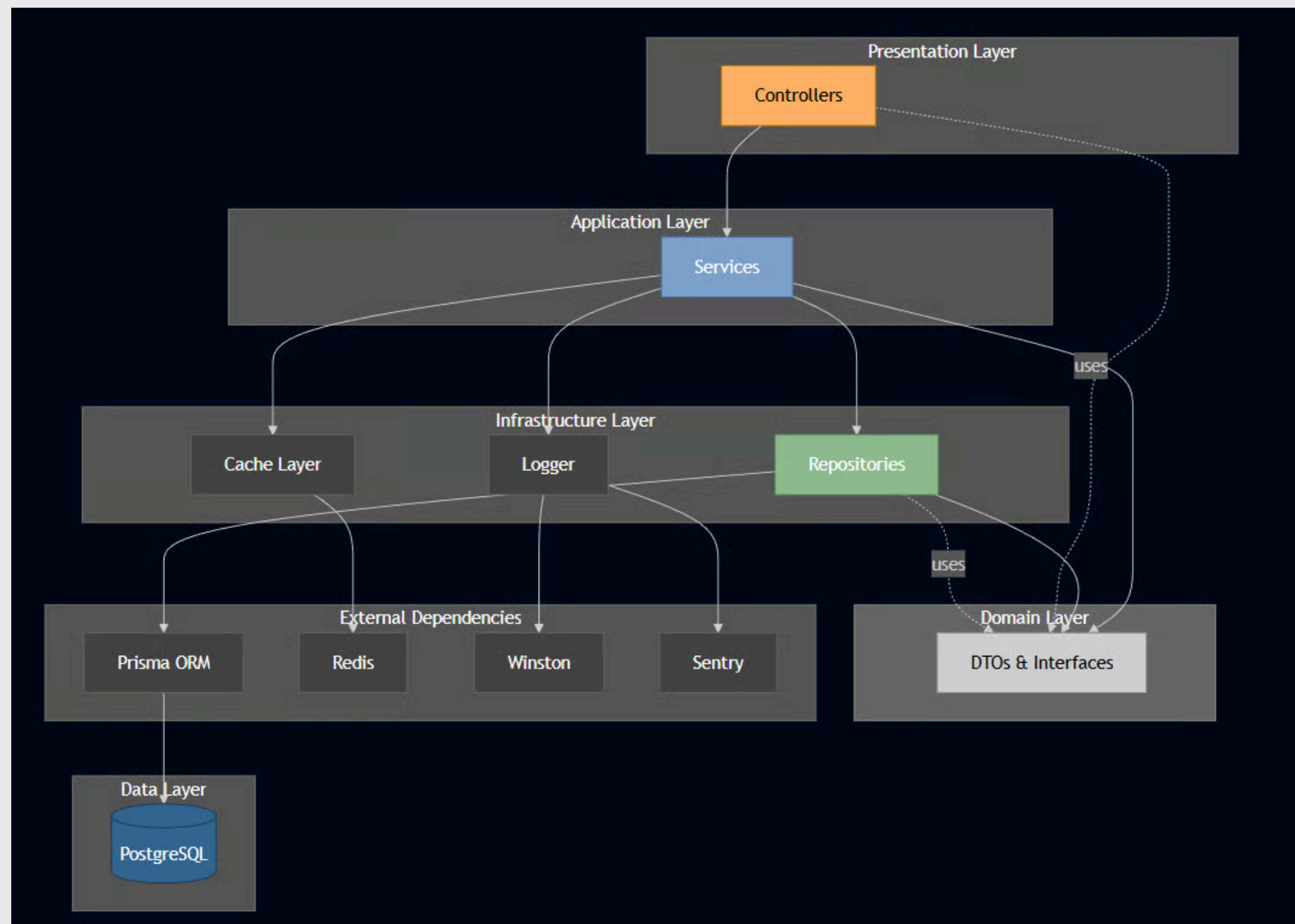
- **WHY:** Needed modern, maintainable SPA for browsing cardiovascular disease content with strong UX foundations and developer experience
- **WHAT:**
 - **React 19 + TypeScript + Vite** with fast HMR
 - **Redux Toolkit** for centralized state management with RTK Query scaffold
 - **Reusable UI components** (Header, Button, Input, Loader, ThemeToggle, LanguageSwitcher)
 - **Dark/Light theming** via SCSS modules and ThemeContext
 - **Language Switching** between English and Russian by i18n library
 - **Error Boundary** for crash handling
 - **react-virtualized + Infinity Scroll** for optimizing the render of any list of cards
 - **Storybook** for component, page documentation
 - **Playwright** for e2e tests

```
import { BrowserRouter } from 'react-router-dom';
import { StoreProvider } from '@app/providers/StoreProvider';
import { ErrorBoundary } from '@app/providers/ErrorBoundary';
import { Router } from '@app/providers/Router';
import './styles/index.scss';

const App = () => {
  return (
    <BrowserRouter>
      <div className="app">
        <ErrorBoundary>
          <StoreProvider>
            <Router />
          </StoreProvider>
        </ErrorBoundary>
      </div>
    </BrowserRouter>
  );
};

export default App;
```

Backend Architecture & API Delivery



- **WHY:** Required production-ready, modular API for cardiovascular disease data with reliability, observability, and bilingual support
- **WHAT:**
 - **Layered architecture** (controllers → services → repositories) with **dependency injection**
 - **REST** endpoints for diseases/sources with pagination, filtering, search
 - **Locale-aware responses** (EN/RU) through Prisma translation tables
 - **Redis caching** for performance optimization
 - **OpenAPI/Swagger** documentation for API consumers

Database

- **WHY:** Needed relational DB for transactional data with complex relations, integrity constraints, and bilingual content support
- **WHAT:**
 - PostgreSQL with 12 tables, 3NF normalized schema
 - M:N junction tables for disease-symptom and disease-risk factor relationships
 - Translation tables keyed by locale (EN/RU) for all content entities
 - Indexes on frequently queried columns (locale fields)
 - Automated migrations via Prisma with versioning
 - Inspired by the analyzed datasets fields and data from them
- **TECH:** PostgreSQL 15, Prisma ORM, migrations, pgAdmin

Observability (Logging + Error Monitoring)

- **WHY:** Required comprehensive monitoring and debugging capabilities across frontend and backend for faster issue resolution and demo reliability
- **WHAT:**
 - **Winston** structured JSON logging with configurable levels (LOG_LEVEL env)
 - **Morgan HTTP request logging** (method, path, status, response time)
 - **Sentry error tracking** for real-time exception monitoring in frontend and backend
 - **Centralized Express error middleware** with automatic logging and Sentry reporting

TECH: Winston, Morgan, Sentry (@sentry/node, @sentry/react), Error Boundary

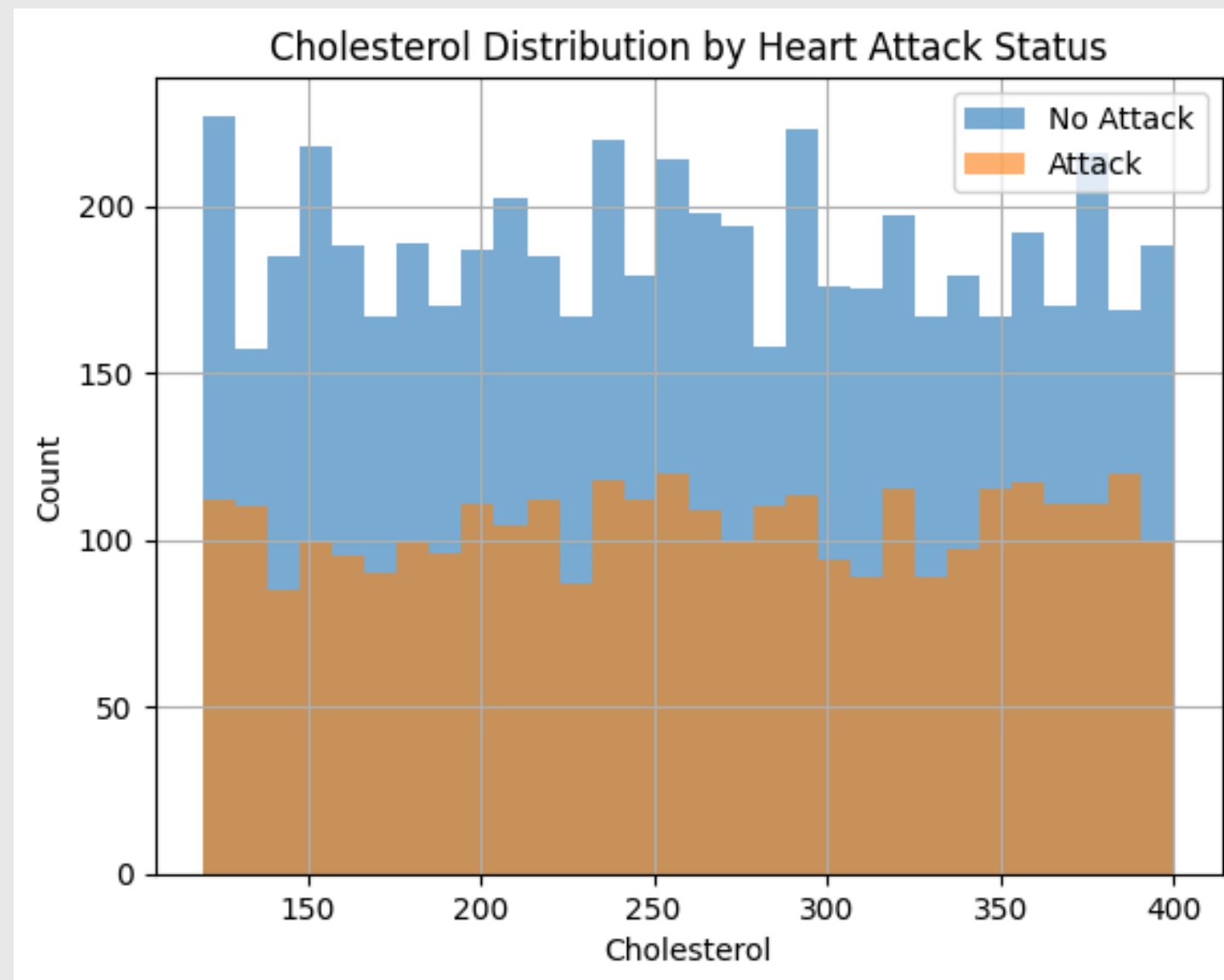
The screenshot displays the Datadog 'Issues' interface. On the left is a sidebar with navigation options: Issues (selected), Errors & Outages, Breached Metrics, Warnings, User Feedback, All Views, Configure, Alerts, Insights, Prevent, and Settings. The main area is titled 'Feed' and shows a list of error events. The table columns are: Issue, Last Seen, Age, Trend, 24h, 14d, Events, Users, Priority, and Assignee. Five error events are listed, each with a title, description, and associated metadata.

Issue	Last Seen	Age	Trend	24h	14d	Events	Users	Priority	Assignee
PrismaClientKnownRequestError Invalid 'prisma.riskFactor.findMany()' invocation: The table 'public.Ri...' CVD-BACKEND-5 GET /api/diseases/risk-factors	4d ago	4d	New			1	1	High	
PrismaClientInitializationError Invalid 'this.prisma.disease.findMany()' invocation in D:\cvd-platform... CVD-BACKEND-2 GET /api/diseases	5d ago	6d	New			3	1	High	
Error EACCES: permission denied, mkdir CVD-BACKEND-4 Unhandled <instance__members__initializer>(utils:logger)	5d ago	5d	New			1	0	High	
ReferenceError ConfigService is not defined CVD-BACKEND-3 Unhandled Object.?(cache:redisClient) Quick Fix	6d ago	6d	New			4	0	High	
Error Sentry testing error CVD-BACKEND-1 Unhandled GET /api/debug-sentry	6d ago	6d	New			1	1	High	

1-5 of 5

Privacy Policy Terms of Use Service Status API Docs Contribute

Research Pipeline & Analytics Artifacts



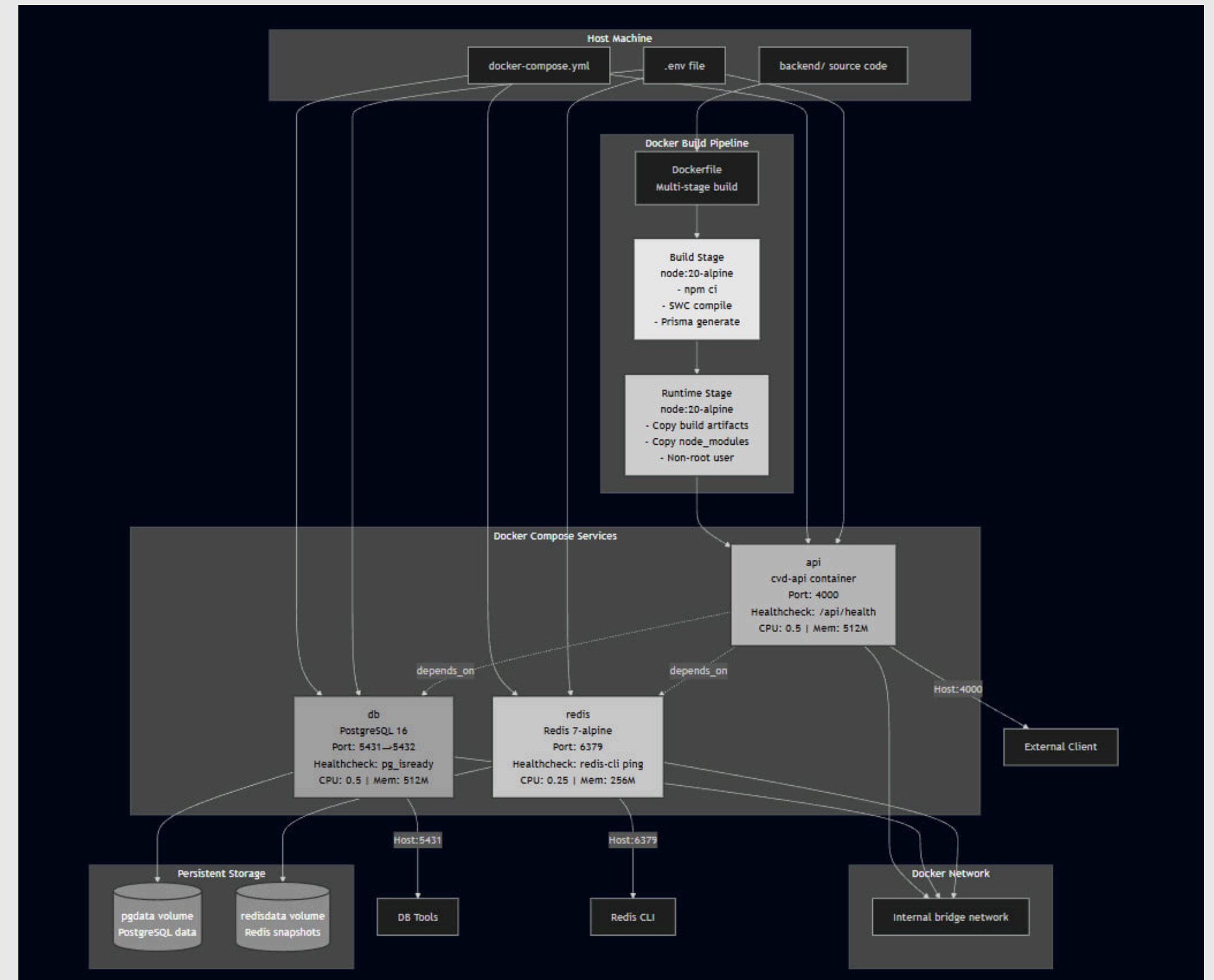
- **WHY:** Add the research about CVDs in order to create database entities based on the analyzed fields and data. Additionally, let the users read interesting information (statistics) about the heart diseases.
- **WHAT:**
 - Analysis of 4 real-world datasets (UCI Heart Disease, Statlog, Cleveland, Hungary) with clinical data
 - Statistical methods: descriptive stats, Welch's t-test, Chi-square, Cohen's d, correlation analysis
 - Python ETL scripts generating static artifacts (SVG/PNG) for web UI embedding
 - Reproducible analysis via committed datasets and Google Colab notebooks
 - Fields analyzed: clinical indicators, lifestyle factors, medical history, demographics
- **TECH:** Python, Pandas, NumPy, Matplotlib, Seaborn, SciPy, Scikit-learn, Jupyter/Colab

Research Types Conducted

- 1. Descriptive Statistics & Visual Data Analysis
 - Data quality assessment (missing values, duplicates)
 - Target distribution analysis (class balance)
 - Univariate analysis (numeric and categorical features)
 - Distribution visualizations (histograms, bar charts)
- 2. Bivariate Statistical Analysis
 - Numeric vs. Target:
 - Welch's t-test (group comparisons)
 - Cohen's d (effect size calculation)
 - Boxplot visualizations
 - Group statistics (mean, median, standard deviation)
 - Categorical vs. Target:
 - Chi-square tests for association
 - Disease/risk rate by category
 - Cross-tabulation analysis
- 3. Correlation & Relationship Analysis
 - Pearson correlation matrices
 - Correlation heatmaps
 - Feature-to-target correlation analysis
 - Inter-feature correlation assessment
- 4. Clinical & Epidemiological Research
 - Disease prevalence analysis
 - Risk factor identification
 - Lifestyle factor associations
 - Clinical indicator validation

Containerization

- **WHY:** Provide reproducible local environment and production-like deployment model for development, testing, and demo purposes
- **WHAT:**
 - Docker Compose orchestration with 3 services (API, PostgreSQL, Redis)
 - Multi-stage Dockerfile reducing API image from ~1.5 GB to 891 MB
 - Named volumes (pgdata, redisdata) for data persistence across restarts
 - Health checks monitoring service availability every 30 seconds
 - Resource limits (CPU: 0.5 cores API/DB, 0.25 Redis; Memory: 512 MB API/DB, 256 MB Redis)
- **TECH:** Docker, Docker Compose, multi-stage builds, health checks, volume management



Testing & Quality Assurance

File	% Stmts	% Branch	% Funcs	% Lines	Uncovered Line #s
All files	93.65	83.51	96.07	94.02	
src	100	100	100	100	
types.ts	100	100	100	100	
src/config	100	100	100	100	
configService.ts	100	100	100	100	
index.ts	100	100	100	100	
keys.ts	100	100	100	100	
src/constants	87.5	100	100	100	
messages.ts	87.5	100	100	100	
src/controllers	92.3	66.66	75	100	
baseController.ts	92.3	66.66	75	100	23
src/disease	90.22	83.05	100	89.92	
disease.controller.ts	96.77	86.95	100	96.77	173,177
disease.repository.ts	82.97	83.87	100	81.39	97,160,183,206-261
disease.service.ts	87.5	60	100	87.5	35,45,68
src/errors	100	87.5	100	100	
ApiError.ts	100	0	100	100	9
createApiError.ts	100	100	100	100	
index.ts	100	100	100	100	
src/middlewares	100	75	100	100	
errorMiddleware.ts	100	75	100	100	22-29
src/source	97.95	96	100	97.91	
source.controller.ts	100	100	100	100	
source.repository.ts	100	100	100	100	
source.service.ts	92.3	75	100	92.3	19
src/utlis	94.44	25	87.5	93.75	
logger.ts	94.44	25	87.5	93.75	9
Test Suites: 9 passed, 9 total					
Tests: 101 passed, 101 total					
Snapshots: 0 total					
Time: 2.07 s, estimated 4 s					

- **WHY:** Prevent regressions and ensure reliability through automated testing at unit, integration, and E2E levels
- **WHAT:**
 - 94.02% line coverage and 93.65% statement coverage across backend
 - Jest unit tests for controllers, services, repositories, config, utilities
 - Playwright E2E tests for critical user flows (error-page, not-found, theme-toggle)
 - Code quality enforcement via ESLint, Prettier, Stylelint, Husky pre-commit hooks
 - Storybook for component visual testing and documentation
- **TECH:** Jest, Playwright, Storybook, ESLint, Prettier, Stylelint, Husky, GitHub Actions

Challenges & Solutions

- **Late Frontend Requirements → Backend-First Prioritization**
 - Problem: Requirements clarified after deadline; frontend routes incomplete
 - Solution: Focused on stabilizing backend architecture and documentation to deliver functional API layer; prioritized normalized data model with bilingual support
 - Learned: Set "end-to-end MVP demo" milestones earlier to prevent UI lagging behind backend
- **Documentation Drift → Implementation Alignment**
 - Problem: Templates easily diverged from actual implementation
 - Solution: Re-checked controllers/OpenAPI specifications against documentation; treated docs as deliverable aligned to actual codebase
 - Learned: Keep documentation close to code changes; formal requirements sign-off checkpoint reduces late scope creep
- **Incomplete Frontend Flows → Technical Debt Tracking**
 - Problem: Frontend routes not fully wired to backend endpoints
 - Solution: Catalogued as TD-001 with severity rating; backend API contracts ready for future frontend completion
- **Learned:** A coherent MVP requires an end-to-end "happy path" early; cutting to "core reading flows" first increases probability of complete UX

Results

- The Cardiovascular Disease Prevention Platform successfully demonstrates the viability of a web-based knowledge system for consolidating fragmented CVD information. The project delivers:
 - A complete backend API with layered architecture, Redis caching, and bilingual support
 - A normalized database schema supporting diseases, symptoms, risk factors, and translations
 - A research pipeline with 4 analyzed datasets covering 20,000+ patient records
 - Production-ready containerization with Docker Compose, health checks, and resource management
 - High code quality with 94%+ test coverage and enforced linting standards
 - Comprehensive observability through structured logging and Sentry error tracking
- The platform provides query-optimized, language-aware access to cardiovascular disease data and establishes a foundation for integration with frontend applications, analytics dashboards, and future microservices expansion. While some frontend features remain incomplete due to timeline constraints, the core knowledge system architecture and data foundation are fully operational and ready for continued development



Q&A

- [Repository Link](#)
- [API Documentation Link](#)
- [Design Link](#)